

1. Status

Phase 4 — Users & Addresses: COMPLETE & VERIFIED.

2. Files Created

1. `src/modules/addresses/types/address.types.ts` — Domain types, DTOs, `SafeAddress`, and `AddressSnapshot` definitions.
2. `src/modules/addresses/models/address.model.ts` — Mongoose `AddressModel` with strict validation, partial unique index, and listing index.
3. `src/modules/addresses/repositories/address.repository.ts` — Owner-scoped persistence methods, atomic default unsetting, and count queries.
4. `src/modules/addresses/services/address.service.ts` — Address business logic, transactional default workflow, and inter-module checkout boundary.
5. `src/modules/addresses/controllers/address.controller.ts` — HTTP controllers with standard response envelopes and privacy-safe logging.
6. `src/modules/addresses/schemas/address.schema.ts` — Strict Zod validation schemas for address creation, update, and `ObjectId` params.
7. `src/modules/addresses/routes/address.routes.ts` — Express router protecting all address endpoints with `requireAuthentication()`.
8. `src/modules/addresses/utils/address.projection.ts` — `toSafeAddress()` and pure `toAddressSnapshot()` utilities.
9. `src/database/migrations/scripts/20260927_002_addresses.migration.ts` — Idempotent migration for addresses collection indexes.
10. `docs/PHASE-4-USERS-ADDRESSES.md` — Comprehensive technical documentation for Phase 4.
11. `tests/unit/address-schema.test.ts` — Unit tests for Zod validation schemas.
12. `tests/unit/address-projection.test.ts` — Unit tests for safe address projection and immutable snapshot generation.
13. `tests/unit/addresses-migration.test.ts` — Unit tests for migration `up()` and `down()` execution.
14. `tests/api/address-crud.test.ts` — Supertest API tests for address CRUD operations and status codes.
15. `tests/api/address-ownership.test.ts` — Supertest API tests for multi-user ownership isolation and enumeration protection.
16. `tests/api/address-default.test.ts` — Supertest API tests for multi-document transactional default management and rollback safety.

3. Files Modified

1. `src/modules/users/types/user.types.ts` — Added `UpdateProfileInput` interface.
2. `src/modules/users/schemas/user.schema.ts` — Implemented strict `updateProfileSchema` with mass-assignment protection and explicit email immutability.
3. `src/modules/users/repositories/users.repository.ts` — Added `updateProfile()` repository method with strict field allowlisting.
4. `src/modules/users/services/users.service.ts` — Added `updateProfile()` service method with phone canonicalization, duplicate collision checks, and safe projection.
5. `src/modules/users/controllers/user.controller.ts` — Implemented `getProfileController` and `updateProfileController`.
6. `src/modules/users/routes/user.routes.ts` — Configured customer profile routes.
7. `src/modules/users/index.ts` — Exported user schemas, controllers, and routes.
8. `src/modules/auth/routes/me.routes.ts` — Added `PATCH /` to `meRouter` using `updateProfileController` and `updateProfileSchema`.

9. `src/modules/addresses/index.ts` — Exported all public types, model, repository, service, controllers, routes, schemas, and utils.
10. `src/app.ts` — Mounted `addressRouter` under `/addresses (/api/v1/addresses)`.
11. `src/database/migrations/index.ts` — Exported `addressesMigration`.
12. `tests/api/me.test.ts` — Expanded test coverage for `PATCH /api/v1/me`, including validation, phone collision, security, and mass-assignment tests.
13. `README.md` — Linked Phase 4 technical documentation.

4. User Module

- **Profile Read (`GET /api/v1/me`):** Retrieves current authenticated customer profile via `userService.getUserById(principal.userId)`. Preserves 100% backward compatibility with Phase 3.
- **Profile Update (`PATCH /api/v1/me`):** Protected by `requireAuthentication()`. Accepts allowlisted fields only.
- **Editable Fields:**
 - `name`: string, trimmed, 2–100 characters. Supports Arabic script (e.g. "أحمد محمود") and Latin script.
 - `phone`: validated format, canonicalized to `+201[0125]XXXXXXX` or international E.164. Checks for uniqueness against existing accounts; throws `409 RESOURCE_CONFLICT` if owned by another user.
- **Immutable Fields:**
 - `id`, `_id`, `role`, `status`, `passwordHash`, `refreshTokenVersion`, `emailVerifiedAt`, `createdAt`, `updatedAt` are strictly prohibited.
 - `email` changes are rejected with `400 VALIDATION_ERROR` ("Email cannot be modified through profile update. Email changes are not permitted").
- **Safe Projection (`toSafeUser`):**
 - Exposes only: `id`, `name`, `email`, `phone`, `role`, `status`, `emailVerifiedAt`, `createdAt`, `updatedAt`.
 - Secrets (`passwordHash`, `refreshTokenVersion`, token hashes) and internal BSON fields (`_id`, `__v`) are never returned.

5. Address Module

- **Schema:** Stored in `addresses` collection with `strict: 'throw'` and `timestamps: true`. Fields: `userId` (ObjectId), `label` (optional string), `recipientName` (required string), `recipientPhone` (required canonical string), `governorate` (required string), `city` (required string), `area` (required string), `street` (required string), `buildingNumber` (required string), `floor` (optional string), `apartment` (optional string), `landmark` (optional string), `notes` (optional string), `isDefault` (boolean, default false), `createdAt` (UTC Date), `updatedAt` (UTC Date). Explicitly contains **no map pins, GPS coordinates, shipping rules, or order snapshot fields**.
- **Validation:** Zod schemas with `.strict()` reject unknown fields, client-provided `userId`, and invalid formats. `buildingNumber` allows free text (e.g. "12A", "3/4", "بدون رقم"). `floor` and `apartment` allow localized free text (e.g. "ground", "3", "أرضي").
- **CRUD:**
 - `GET /api/v1/addresses`: Lists authenticated user's addresses sorted by `isDefault DESC`, `createdAt DESC`.
 - `POST /api/v1/addresses`: Creates an address assigned strictly to authenticated `userId`, returning `201 Created`.
 - `GET /api/v1/addresses/:id`: Returns single address after ownership verification.
 - `PATCH /api/v1/addresses/:id`: Updates allowlisted fields after ownership verification.
 - `DELETE /api/v1/addresses/:id`: Physically deletes address after ownership verification, returning `204 No Content`.
- **Ownership:** All database operations are scoped at the repository level with `findByIdAndUserId(addressId, authenticatedUserId)`. Access to unowned addresses returns `404 NOT_FOUND` to protect against ID enumeration.
- **Default Address & Transaction Behavior:**
 - When `isDefault: true` is set (on create or update), the operation executes inside `withTransaction()`. Existing defaults for the customer are atomically flipped to `isDefault: false` before persisting the target address.

- If any failure occurs during transaction execution, the transaction rolls back cleanly without leaving an inconsistent default state.
- Database partial unique index `{ userId: 1, isDefault: 1 }` with `partialFilterExpression: { isDefault: true }` guarantees at the storage engine level that no user can have multiple default addresses.
- When addresses are created with `isDefault: false`, no multi-document transaction is triggered, allowing multiple non-default addresses to coexist.

6. API Endpoints

Method	Route	Auth Requirement	Success Status	Description
GET	<code>/api/v1/me</code>	<code>requireAuthentication()</code>	200 OK	Read current user's profile
PATCH	<code>/api/v1/me</code>	<code>requireAuthentication()</code>	200 OK	Update profile (name , phone)
GET	<code>/api/v1/addresses</code>	<code>requireAuthentication()</code>	200 OK	List customer's saved addresses
POST	<code>/api/v1/addresses</code>	<code>requireAuthentication()</code>	201 Created	Create new saved address
GET	<code>/api/v1/addresses/:id</code>	<code>requireAuthentication()</code>	200 OK	Get single saved address
PATCH	<code>/api/v1/addresses/:id</code>	<code>requireAuthentication()</code>	200 OK	Update saved address
DELETE	<code>/api/v1/addresses/:id</code>	<code>requireAuthentication()</code>	204 No Content	Delete saved address

7. Security

- **Ownership Enforcement:** All address queries filter by `userId === authenticatedPrincipal.userId`. Client-provided `userId` is rejected.
- **Mass-Assignment Protection:**
 - Strict Zod schemas reject unexpected or server-controlled properties (`role` , `status` , `passwordHash` , `refreshTokenVersion` , `emailVerifiedAt` , `userId` , `_id`).
 - Strict Mongoose options (`strict: 'throw'`) prevent database-level schema drift.
 - Service layers explicitly map and sanitize update DTOs.
- **ID Validation:** Validates 24-character hexadecimal MongoDB ObjectIds at the HTTP boundary via `addressIdParamSchema`; invalid IDs return `400 VALIDATION_ERROR` rather than leaking internal `CastError` details.
- **Role / Status Protection:** Customer principals cannot self-promote to `admin` / `owner` or alter their account status (`active` / `suspended`).
- **Sensitive-Data Protection:** Responses never contain password hashes, refresh token versions, or token hashes. Internal `_id` and `__v` are replaced by clean string `id`.
- **Logging Rules:** Full address text, recipient phone numbers, and raw customer delivery notes are never logged. Logs record only safe structured metadata: `requestId` , `userId` , `addressId` , `operation` , and `status`.

8. Database

- **Collections Touched:**

- users : Profile reads and updates.
- addresses : Saved address records.
- **Indexes:**
 - addresses (userId, isDefault) : Partial unique index ({ unique: true, partialFilterExpression: { isDefault: true } }).
 - addresses (userId, createdAt: -1) : Compound index for sorted customer listing.
- **Migrations:**
 - 20260927_002_addresses : Idempotent and non-destructive index creation using the existing migration infrastructure (src/database/migrations/).
- **Transaction Boundaries:**
 - Setting or switching a default address utilizes withTransaction() multi-document transactions across the replica set.
 - Single-document reads and non-default creations execute without unnecessary transaction overhead.
 - External network calls (SMTP, Cloudinary, etc.) are never executed inside MongoDB transactions.

9. Tests

Total test files in suite: **39 files** Total passing tests: **297 tests** (100% pass rate)

Phase 4 Test Breakdown:

- **Profile Security & CRUD Tests (tests/api/me.test.ts): 19 tests**
 - Authenticated and unauthenticated GET /me
 - Name updates (Latin and Arabic script)
 - Phone updates and canonicalization
 - Duplicate phone rejection (409 Conflict)
 - Email immutability rejection (400 Validation)
 - Mass assignment rejection (role , status , passwordHash , refreshTokenVersion , id)
 - Suspended user rejection (401)
- **Address CRUD API Tests (tests/api/address-crud.test.ts): 15 tests**
 - Address creation (201 Created)
 - User ID injection rejection
 - Unknown field rejection (no map pins, no shipping fields)
 - Address listing with default-first ordering
 - Address retrieval by ID
 - Address updating and empty payload rejection
 - Physical deletion (204 No Content)
 - Invalid ObjectId validation (400)
- **Ownership Isolation Tests (tests/api/address-ownership.test.ts): 12 tests**
 - User A access to Address A (ALLOWED)
 - User A access/patch/delete of User B's Address B (DENIED - 404 safe enumeration protection)
 - User B access to Address B (ALLOWED)
 - User B access/patch/delete of User A's Address A (DENIED - 404 safe enumeration protection)
 - Admin access to customer addresses through customer endpoints (DENIED - 404 safe isolation)
- **Default Address & Transaction Tests (tests/api/address-default.test.ts): 6 tests**
 - Unsetting previous default on creation of new default
 - Unsetting previous default on update to isDefault: true
 - Coexistence of multiple non-default addresses
 - Isolation of User A's default transitions from User B's default address

- Transaction abort and rollback on simulated error
- Partial unique index enforcement at database engine layer
- **Address Schemas & Snapshot Unit Tests** (`tests/unit/address-schema.test.ts` , `tests/unit/address-projection.test.ts`): **17 tests**
 - Zod validation rules (required fields, Arabic strings, phone formats, strict unknown field rejection)
 - Safe projection formatting
 - Pure, deterministic snapshot generation (`toAddressSnapshot`)
- **Addresses Migration Test** (`tests/unit/addresses-migration.test.ts`): **2 tests**
 - Idempotent `up()` index creation and `down()` index cleanup

10. Verification

- `npm run typecheck` : **PASS** (TypeScript strict mode, 0 errors)
- `npm run lint` : **PASS** (ESLint, 0 errors, 0 warnings)
- `npm run build` : **PASS** (compiled cleanly to `dist/`)
- `npm test` : **PASS** (39 test suites, 297 tests passing)
- `npm run dev` : **PASS** (server boots cleanly, connects to MongoDB Atlas)
- `GET /health/live` : **PASS** (200 OK , { "status": "ok" })
- `GET /health/ready` : **PASS** (200 OK , { "status": "ready", "database": "connected" })

11. Database Safety Confirmation

- **No destructive database operations:** `dropDatabase()` , `drop()` , `deleteMany()` , `deleteOne()` , or `updateMany()` were NEVER executed against the real database.
- **No fake users or fake addresses** were created in the real database.
- **No unrelated collections** were modified.
- All test suites run in hermetic isolation using `mongodb-memory-server` replica sets.

12. Open / Deferred Items

1. **Email Change Workflow:** Security-sensitive; requires email re-verification and token handling. Open/deferred; `PATCH /me` rejects email updates with 400 .
2. **Phone OTP Verification:** Deferred / not required per PRD AUTH-006.
3. **Customer Account Self-Deletion:** Deferred to future account lifecycle specifications.
4. **Checkout Address Snapshot Persistence:** Deferred to Phase 8 (Orders & Checkout). Pure helper `toAddressSnapshot()` is provided.
5. **Shipping Cost & Serviceability Calculation:** Deferred to Phase 10 (Shipping & Coupons).

13. Regression Status

- Phase 0 Foundation tests: **PASS**
- Phase 1 Database connectivity & migrations tests: **PASS**
- Phase 2 Common infrastructure, rate limits & HTTP security tests: **PASS**
- Phase 3 Authentication, identity & session management tests: **PASS**

- Phase 4 Users & addresses tests: **PASS**

All 39 test suites and 297 tests pass with zero regressions.

14. Next Phase

Phase 4 complete. Implementation stopped here. No Phase 5 work was implemented.